

EECS 348 Assignment 3 - GenAI Code Analysis

Student: Zareb Rahman

Course: EECS 348 - Software Engineering I

Assignment: Assignment 3 - Object-Oriented Email Priority Queue

Date: October 1, 2026

1. GenAI Programs and Access

GenAI 1: ChatGPT (OpenAI). Accessed through ChatGPT in a web browser.

GenAI 2: Google Gemini. Accessed through Gemini in a web browser.

2. Prompt Used for Both GenAIs

The exact same prompt was used for both systems:

Generate a complete C++ program for EECS 348 Assignment 3 using object-oriented programming. The program must prioritize emails for a CEO using a MaxHeap implemented manually with a list-based structure such as a vector. Do not use `std::priority_queue` or any pre-existing heap functions/modules.

Each EMAIL command has this format:

EMAIL <sender category>,<subject line>,<date>

Sender category priority from highest to lowest:

1. Boss
2. Subordinate
3. Peer
4. ImportantPerson
5. OtherPerson

If two emails have the same sender category, the newest date must have higher priority. Dates are MM-DD-YYYY. Subject lines may contain spaces but not commas.

Commands:

- NEXT: display the highest-priority unread email without removing it, using exactly:

Next email:

Sender: <sender>

Subject: <subject>

Date: <date>

- READ: remove the highest-priority unread email without displaying it.

- COUNT: display the unread count using: There are <count> emails to read.

The grader may use NEXT or READ when the heap is empty, two NEXT commands in a row without READ, or two READ commands in a row without NEXT, so handle all of those safely.

The program must read commands from a test file whose filename is passed as a command-line argument. Use classes/objects, create the MaxHeap from scratch, and make the code easy for a beginning C++ student to understand. Include a full prologue comment with program name, description, input, output, collaborators, sources, author, creation date, revision date, and revisions. Add detailed comments for every line or small group of lines, and identify the source/author of each major code block.

3. Correctness Comparison

Both programs implement a custom MaxHeap backed by `std::vector` rather than `std::priority_queue` or a pre-built heap module. Both prioritize sender categories in the required order: Boss, Subordinate, Peer, ImportantPerson, and OtherPerson. Both then use the date so that newer messages win ties within the same sender category.

Both programs were compiled locally with g++ using C++17 and were tested with the assignment sample file. Both produced the required sample output: six unread emails at first, then the Boss email dated 01-03-2025, then the Boss

email dated 12-20-2024, and finally a count of four unread emails.

The ChatGPT version separates command processing into an **EmailProgram** class, while the Gemini version processes commands directly inside `main()`. The ChatGPT version also makes `READ` on an empty heap a no-op and `NEXT` on an empty heap safe. The Gemini version is also safe, but it prints extra messages for empty `NEXT` and `READ` operations. Because the assignment does not specify exact output for those empty cases, avoiding unnecessary output can be safer for an automated grader.

The Gemini version has a separate **Date** class and overloaded comparison operators. This is valid object-oriented design. The ChatGPT version instead converts the date once when an `Email` object is created and stores the numeric value. Both approaches correctly compare MM-DD-YYYY dates for the assignment assumptions.

4. Execution Time Comparison

For both programs, inserting an `EMAIL` command into the heap takes $O(\log n)$ time because the new item may move upward through the heap.

A `READ` operation takes $O(\log n)$ time because removing the root can require heapifying downward. `NEXT` takes $O(1)$ time because the highest-priority email is stored at index 0. `COUNT` also takes $O(1)$ time because vector size is available directly.

If there are E email insertions and R successful removals, the heap work is approximately $O((E + R) \log E)$, in addition to the time required to read and parse the input text. The two programs therefore have the same asymptotic execution-time complexity.

5. Space Complexity Comparison

Both programs store every unread email in a vector-backed heap. If n emails are currently unread, both use $O(n)$ heap storage.

The extra temporary variables used during parsing and heap operations are constant-size relative to n , so they do not change the overall $O(n)$ space complexity. The Gemini version also stores a `Date` object inside each `Email`, while the ChatGPT version stores one numeric date value. This is only a constant-factor difference, not an asymptotic difference.

6. Maintainability Comparison

The ChatGPT version uses three main classes: **Email** for email data and priority information, **MaxHeap** for the heap implementation, and **EmailProgram** for file parsing and command processing. This keeps `main()` very small and gives each class a clear responsibility.

The Gemini version uses **Date**, **Email**, and **MaxHeap** classes, which is also organized, but most command parsing and output logic remains inside `main()`. Its overloaded operators make comparisons concise, although they may be slightly harder for a beginning C++ student to trace.

Both versions contain detailed comments. The ChatGPT version was selected as easier to maintain for this assignment because the input-processing logic is separated from `main()`, the priority comparison is centralized in `MaxHeap`, and the code structure is straightforward to modify without changing unrelated parts.

7. Selected Baseline and Justification

I selected the **ChatGPT version** as the basis for the final assignment. Both programs were correct on the provided sample and have the same Big-O performance. I chose the ChatGPT version because its `EmailProgram` class gives the program clearer separation of responsibilities, its `main()` function is simpler, and its empty `READ` behavior avoids

printing extra output that was not requested. These differences make it a stronger fit for the assignment and easier to understand and maintain.

8. Improvements Made to the Selected Code

Correctness: Verified the sender-category ordering and date tie-break rule, tested the provided sample, kept NEXT from removing an email, kept READ from displaying an email, and made empty READ safe. Added removal of a trailing carriage return so Windows-style CRLF test files still match NEXT, READ, and COUNT correctly when the program runs on Linux.

Execution time: Kept the manual binary MaxHeap so insertion and READ remain $O(\log n)$, while NEXT and COUNT remain $O(1)$. The date is converted to YYYYMMDD once when an Email object is created instead of reparsing it during each heap comparison.

Space complexity: Kept one vector of unread Email objects, so space remains $O(n)$. No duplicate heap or unnecessary secondary list was added.

Maintainability: Separated responsibilities among Email, MaxHeap, and EmailProgram. Added descriptive names, a complete prologue, detailed line/block comments, and source notes for major code blocks. The final code avoids pre-built heap functions and keeps the heap logic in one class.

9. Final Complexity Summary

Operation	ChatGPT final	Gemini
EMAIL insert	$O(\log n)$	$O(\log n)$
NEXT	$O(1)$	$O(1)$
READ	$O(\log n)$	$O(\log n)$
COUNT	$O(1)$	$O(1)$
Heap storage	$O(n)$	$O(n)$

Local verification note: both generated programs were compiled with `g++ -std=c++17` and produced the required output for the provided sample test file. The course instructions require the final submitted executable to be compiled on the EECS Cycle server or a lab machine; that environment-specific executable should be added when access is available.

Appendix A - Raw ChatGPT Code

The following is the complete final ChatGPT-based source code used as the assignment baseline after the documented improvements.

```
/*
 * Program: EECS 348 Assignment 3 - CEO Email Priority Queue
 * Description: Reads email commands from a test file and stores unread emails in a
 *              list-based MaxHeap. Emails are prioritized by sender category first
 *              and newest date second. NEXT displays the highest-priority email,
 *              READ removes it, and COUNT displays the unread-email count.
 * Input: A text file containing EMAIL, NEXT, READ, and COUNT commands.
 * Output: Terminal output showing the next email and/or number of unread emails.
 * Collaborators: None.
 * Other sources: ChatGPT (OpenAI) was used to generate and revise the initial code.
 * Author: Zareb Rahman
 * Creation date: October 1, 2026
 * Revision date: October 1, 2026
 * Revisions: Added object-oriented design, list-based MaxHeap, input parsing,
 *            edge-case handling, and detailed comments.
 */

// Include input/output tools for terminal messages and file reading.
#include <iostream>
// Include file-stream tools so the program can read the test file.
#include <fstream>
// Include strings for sender names, subjects, dates, and input lines.
#include <string>
// Include vectors to provide the list used to store the MaxHeap.
#include <vector>
// Include string streams to split EMAIL lines at commas.
#include <sstream>

// Use standard-library names without writing std:: before each one.
using namespace std;

/*
 * Source note for this block:
 * The Email class was generated with assistance from ChatGPT and reviewed/revised
 * by the author. It stores one email and knows how to calculate its own priority.
 */
class Email {
private:
    // Store the sender category, such as Boss or Peer.
    string sender;
    // Store the email subject line.
    string subject;
    // Store the original date text in MM-DD-YYYY format.
    string date;
    // Store the date as YYYYMMDD so newer dates have larger numbers.
    int dateValue;

    // Convert a date from MM-DD-YYYY into an integer in YYYYMMDD order.
    int convertDate(const string& dateText) const {
        // Read the two-digit month from the start of the string.
        int month = stoi(dateText.substr(0, 2));
        // Read the two-digit day from the middle of the string.
        int day = stoi(dateText.substr(3, 2));
        // Read the four-digit year from the end of the string.
        int year = stoi(dateText.substr(6, 4));

        // Return a number that sorts correctly from oldest to newest.
        return year * 10000 + month * 100 + day;
    }
}

public:
    // Create an empty email so temporary Email objects can be made when needed.
    Email() : sender(""), subject(""), date(""), dateValue(0) {}

    // Create a complete email from its sender, subject, and date.
    Email(const string& newSender, const string& newSubject, const string& newDate)
        : sender(newSender), subject(newSubject), date(newDate), dateValue(convertDate(newDate)) {}

    // Return the sender category.
    string getSender() const {
        return sender;
    }

    // Return the subject line.
    string getSubject() const {
        return subject;
    }

    // Return the original date string.
    string getDate() const {
        return date;
    }

    // Return the numeric date used for comparisons.
```

```

int getDateValue() const {
    return dateValue;
}

// Return the category priority, where a larger number means higher priority.
int getCategoryPriority() const {
    // Boss emails have the highest category priority.
    if (sender == "Boss") {
        return 5;
    }
    // Subordinate emails are second-highest.
    if (sender == "Subordinate") {
        return 4;
    }
    // Peer emails are third-highest.
    if (sender == "Peer") {
        return 3;
    }
    // ImportantPerson emails are fourth-highest.
    if (sender == "ImportantPerson") {
        return 2;
    }
    // OtherPerson emails have the lowest category priority.
    return 1;
}
};

/*
 * Source note for this block:
 * The MaxHeap class was generated with assistance from ChatGPT and reviewed/revised
 * by the author. It implements the heap manually with a vector and does not use
 * std::priority_queue, make_heap, push_heap, pop_heap, or another heap module.
 */
class MaxHeap {
private:
    // Store the heap in a list-like vector.
    vector<Email> heap;

    // Return true when the first email should be above the second email in the heap.
    bool higherPriority(const Email& first, const Email& second) const {
        // Compare sender categories before dates.
        if (first.getCategoryPriority() != second.getCategoryPriority()) {
            // The email with the larger category value has higher priority.
            return first.getCategoryPriority() > second.getCategoryPriority();
        }
        // When categories match, the newer date has higher priority.
        return first.getDateValue() > second.getDateValue();
    }

    // Move a newly inserted email upward until the MaxHeap property is restored.
    void heapifyUp(int index) {
        // Keep checking parents while the item is not at the root.
        while (index > 0) {
            // Calculate the parent index in a zero-based list.
            int parent = (index - 1) / 2;
            // Stop when the parent already has equal or higher priority.
            if (!higherPriority(heap[index], heap[parent])) {
                break;
            }
            // Swap the child and parent because the child has higher priority.
            Email temp = heap[index];
            // Move the parent down to the child position.
            heap[index] = heap[parent];
            // Move the higher-priority email up to the parent position.
            heap[parent] = temp;
            // Continue checking from the email's new location.
            index = parent;
        }
    }

    // Move the root downward after a READ removes the highest-priority email.
    void heapifyDown(int index) {
        // Continue until the current item is in the correct position.
        while (true) {
            // Calculate the left child index.
            int left = 2 * index + 1;
            // Calculate the right child index.
            int right = 2 * index + 2;
            // Start by assuming the current item has the highest priority.
            int largest = index;

            // Check whether the left child exists and has higher priority.
            if (left < static_cast<int>(heap.size()) && higherPriority(heap[left], heap[largest])) {
                // Remember that the left child is currently the best choice.
                largest = left;
            }

            // Check whether the right child exists and has higher priority.
            if (right < static_cast<int>(heap.size()) && higherPriority(heap[right], heap[largest])) {

```

```

        // Remember that the right child is the best choice.
        largest = right;
    }

    // Stop when neither child has higher priority than the current item.
    if (largest == index) {
        break;
    }

    // Temporarily save the current email before swapping.
    Email temp = heap[index];
    // Move the higher-priority child upward.
    heap[index] = heap[largest];
    // Move the old current email downward.
    heap[largest] = temp;
    // Continue checking from the lower position.
    index = largest;
}

}

public:
    // Add one email to the heap.
    void insert(const Email& email) {
        // Put the new email at the end of the list.
        heap.push_back(email);
        // Move it upward if it has more priority than its parent.
        heapifyUp(static_cast<int>(heap.size()) - 1);
    }

    // Return true when no unread emails remain.
    bool empty() const {
        return heap.empty();
    }

    // Return the number of unread emails.
    int size() const {
        return static_cast<int>(heap.size());
    }

    // Return the highest-priority email without removing it.
    const Email& peek() const {
        return heap[0];
    }

    // Remove the highest-priority email if one exists.
    void removeMax() {
        // Do nothing if there are no unread emails.
        if (heap.empty()) {
            return;
        }

        // Replace the root with the last email in the list.
        heap[0] = heap.back();
        // Remove the old last position from the list.
        heap.pop_back();

        // Restore the heap only when at least one email remains.
        if (!heap.empty()) {
            // Move the new root downward to its correct position.
            heapifyDown(0);
        }
    }
};

/*
 * Source note for this block:
 * The EmailProgram class was generated with assistance from ChatGPT and revised by
 * the author. It handles all file parsing and command processing through objects.
 */
class EmailProgram {
private:
    // Store all unread emails inside the custom MaxHeap object.
    MaxHeap inbox;

    // Process one EMAIL command and add the parsed email to the heap.
    void processEmail(const string& line) {
        // Remove the "EMAIL " part from the beginning of the command.
        string data = line.substr(6);
        // Create a stream that can split the remaining text by commas.
        stringstream ss(data);
        // Create a variable for the sender category.
        string sender;
        // Create a variable for the subject line.
        string subject;
        // Create a variable for the date.
        string date;

        // Read everything before the first comma as the sender category.
        getline(ss, sender, ',');

```

```

    // Read everything before the second comma as the subject line.
    getline(ss, subject, ',');
    // Read the rest of the line as the date.
    getline(ss, date);

    // Create an Email object from the three parsed values.
    Email email(sender, subject, date);
    // Add the Email object to the custom MaxHeap.
    inbox.insert(email);
}

// Process a NEXT command without removing the email.
void processNext() const {
    // Handle NEXT safely when the inbox is empty.

    if (inbox.empty()) {
        // Tell the user that there is no email available to display.
        cout << "No emails to read." << endl;
        // End this command without accessing the empty heap.
        return;
    }

    // Get the highest-priority email without removing it.
    const Email& email = inbox.peek();
    // Print the heading required by the sample output.
    cout << "Next email:" << endl;
    // Print the sender category.
    cout << "Sender: " << email.getSender() << endl;
    // Print the subject line.
    cout << "Subject: " << email.getSubject() << endl;
    // Print the date.
    cout << "Date: " << email.getDate() << endl;
}

// Process a READ command by removing the highest-priority email.
void processRead() {
    // removeMax safely does nothing when the heap is empty.
    inbox.removeMax();
}

// Process a COUNT command by displaying the number of unread emails.
void processCount() const {
    // Print the unread-email count in the format shown in the assignment sample.
    cout << "There are " << inbox.size() << " emails to read." << endl;
}

public:
    // Read and process every command in the given test file.
    bool run(const string& fileName) {
        // Open the input file for reading.
        ifstream inputFile(fileName);

        // Check whether the file opened successfully.
        if (!inputFile.is_open()) {
            // Print an error message if the file could not be opened.
            cerr << "Error: Could not open " << fileName << endl;
            // Tell main that the program failed.
            return false;
        }

        // Store one input line at a time.
        string line;
        // Continue until every line in the file has been processed.
        while (getline(inputFile, line)) {
            // Remove a trailing carriage return so Windows-style test files work on Linux.
            if (!line.empty() && line.back() == '\r') {
                line.pop_back();
            }

            // Ignore completely empty lines.
            if (line.empty()) {
                continue;
            }

            // Check whether this line is an EMAIL command.
            if (line.rfind("EMAIL ", 0) == 0) {
                // Parse and insert the email.
                processEmail(line);
            }
            // Check whether this line is a NEXT command.
            else if (line == "NEXT") {
                // Display the current highest-priority email without removing it.
                processNext();
            }
            // Check whether this line is a READ command.
            else if (line == "READ") {
                // Remove the current highest-priority email without displaying it.
                processRead();
            }
        }
    }
}

```

```

        // Check whether this line is a COUNT command.
        else if (line == "COUNT") {
            // Display the number of unread emails.
            processCount();
        }
    }

    // Close the file after all commands have been processed.
    inputFile.close();
    // Tell main that the program completed successfully.
    return true;
}

};

/*
 * Source note for this block:
 * main was generated with assistance from ChatGPT and reviewed by the author.
 * It creates the program object and starts processing the selected test file.
 */
int main(int argc, char* argv[]) {
    // Require the test-file name as a command-line argument.
    if (argc != 2) {
        // Show the expected command format when the file name is missing.
        cerr << "Usage: " << argv[0] << " <test_file>" << endl;
        // Exit with an error status.
        return 1;
    }

    // Create the object that controls the email-priority program.
    EmailProgram program;
    // Run the program using the file name supplied on the command line.
    if (!program.run(argv[1])) {
        // Exit with an error status if the file could not be processed.
        return 1;
    }

    // Exit successfully after the entire test file is complete.
    return 0;
}

```

Appendix B - Raw Google Gemini Code

The following is the complete raw code supplied by Google Gemini for comparison.

```
/*
 * Program Name: EECS 348 Assignment 3 - Email Priority Queue
 * Description: Implements an object-oriented email prioritization system for a CEO
 *              using a custom list-based Max-Heap (std::vector). Reads commands from
 *              a file passed via command-line arguments and handles EMAIL, NEXT,
 *              READ, and COUNT instructions safely.
 * Input: Filename via command-line argument containing sequential commands.
 * Output: Console output for NEXT commands, COUNT queries, and edge case notifications.
 * Collaborators: None
 * Sources: EECS 348 Lecture Notes on Binary Heap Data Structures & C++ I/O Stream Docs
 * Author: EECS 348 Student
 * Creation Date: March 30, 2026
 * Revision Date: October 1, 2026
 * Revisions:
 *   - 03/30/2026: Initial setup of Email, Date, and MaxHeap classes.
 *   - 03/31/2026: Implemented heapifyUp and heapifyDown algorithms.
 *   - 10/01/2026: Fixed edge cases for empty heap reads and added detailed line comments.
 */

#include <iostream> // For standard input/output operations (std::cout, std::cerr)
#include <fstream> // For file stream operations (std::ifstream)
#include <string> // For string manipulation (std::string, std::getline)
#include <vector> // For dynamic array list implementation in MaxHeap
#include <sstream> // For string parsing operations (std::stringstream)

// Source: Custom Class Definition by Author
// Class representing a calendar date formatted as MM-DD-YYYY
class Date {
private:
    int month; // Month (1-12)
    int day; // Day (1-31)
    int year; // Year (YYYY)

public:
    // Default Constructor initializing date to zero
    Date() : month(0), day(0), year(0) {}

    // Constructor parsing date string formatted as "MM-DD-YYYY"
    Date(const std::string& dateStr) {
        // Stringstream helps parse integers around the hyphen delimiters
        std::stringstream ss(dateStr);
        char dash1, dash2;
        ss >> month >> dash1 >> day >> dash2 >> year;
    }

    // Helper method to convert the date into a single integer YYYYMMDD for easy comparisons
    int toNumericValue() const {
        return (year * 10000) + (month * 100) + day;
    }

    // Overloaded strictly-greater-than operator for date comparison
    // Returns true if this date is NEWER (larger numeric value) than other date
    bool operator>(const Date& other) const {
        return this->toNumericValue() > other.toNumericValue();
    }

    // Overloaded equality operator
    bool operator==(const Date& other) const {
        return this->toNumericValue() == other.toNumericValue();
    }
};

// Source: Custom Class Definition by Author
// Class representing an individual Email
class Email {
private:
    std::string senderCategory; // Category: Boss, Subordinate, Peer, ImportantPerson, OtherPerson
    std::string subject; // Email subject line
    std::string rawDateStr; // Original date string format (MM-DD-YYYY)
    Date date; // Date object for chronological comparison

    // Helper method mapping category strings to priority ranks (Higher integer = Higher priority)
    int getCategoryPriority() const {
        if (senderCategory == "Boss") return 5;
        if (senderCategory == "Subordinate") return 4;
        if (senderCategory == "Peer") return 3;
        if (senderCategory == "ImportantPerson") return 2;
        if (senderCategory == "OtherPerson") return 1;
        return 0; // Fallback default for unknown categories
    }

public:
    // Default Constructor
    Email() = default;
```

```

// Parameterized Constructor
Email(const std::string& category, const std::string& subj, const std::string& dStr)
    : senderCategory(category), subject(subj), rawDateStr(dStr), date(dStr) {}

// Getters for email properties
std::string getSenderCategory() const { return senderCategory; }
std::string getSubject() const { return subject; }
std::string getRawDateStr() const { return rawDateStr; }

// Overloaded greater-than operator defining total order for Priority Queue
// Priority Rule 1: Sender Category (Boss > Subordinate > Peer > ImportantPerson > OtherPerson)
// Priority Rule 2: Newer Date wins ties within the same sender category
bool operator>(const Email& other) const {
    int myRank = this->getCategoryPriority();
    int otherRank = other.getCategoryPriority();

    // If categories are different, compare by category rank
    if (myRank != otherRank) {
        return myRank > otherRank;
    }

    // If categories are equal, break tie using date (newer date = higher priority)
    return this->date > other.date;
}

// Overloaded less-than operator (convenience helper)
bool operator<(const Email& other) const {
    return other > *this;
}
};

// Source: Custom MaxHeap Implementation by Author
// Class representing a custom Max-Heap built on top of std::vector
class MaxHeap {
private:
    std::vector<Email> heap; // Underlying list container storing heap elements

    // Parent index formula in dynamic array binary tree representation
    int parent(int i) const { return (i - 1) / 2; }

    // Left child index formula
    int leftChild(int i) const { return (2 * i) + 1; }

    // Right child index formula
    int rightChild(int i) const { return (2 * i) + 2; }

    // Restores heap order upward after inserting a new element at the end
    void heapifyUp(int index) {
        // Continue bubbling up as long as not at root and current element > parent
        while (index > 0 && heap[index] > heap[parent(index)]) {
            std::swap(heap[index], heap[parent(index)]); // Swap current with parent
            index = parent(index); // Move index up to parent
        }
    }

    // Restores heap order downward after removing the root element
    void heapifyDown(int index) {
        int maxIndex = index;
        int left = leftChild(index);
        int right = rightChild(index);
        int size = heap.size();

        // Compare current largest with left child
        if (left < size && heap[left] > heap[maxIndex]) {
            maxIndex = left;
        }

        // Compare current largest with right child
        if (right < size && heap[right] > heap[maxIndex]) {
            maxIndex = right;
        }

        // If index is not the largest, swap with largest child and recurse/loop down
        if (index != maxIndex) {
            std::swap(heap[index], heap[maxIndex]);
            heapifyDown(maxIndex); // Downward percolation continues
        }
    }

public:
    // Default Constructor
    MaxHeap() = default;

    // Returns true if the heap contains no elements
    bool isEmpty() const {
        return heap.empty();
    }

    // Returns the current number of elements in the heap

```

```

int size() const {
    return heap.size();
}

// Inserts a new email into the Max-Heap
void insert(const Email& email) {
    heap.push_back(email);          // Place new element at end of dynamic array
    heapifyUp(heap.size() - 1);     // Restore heap order upward
}

// Returns the highest priority email without removing it
Email peek() const {
    if (isEmpty()) {
        throw std::runtime_error("Heap is empty!");
    }
    return heap[0]; // Root element is always highest priority
}

// Removes and returns the highest priority email from the Max-Heap
Email extractMax() {
    if (isEmpty()) {
        throw std::runtime_error("Heap is empty!");
    }

    Email maxEmail = heap[0];        // Save top priority element
    heap[0] = heap.back();          // Move last element to root
    heap.pop_back();                // Remove last element

    if (!isEmpty()) {
        heapifyDown(0);              // Restore heap order downward from root
    }

    return maxEmail;
}
};

// Source: File Command Parser and Main Function by Author
int main(int argc, char* argv[]) {
    // Validate that input filename is provided as command-line argument
    if (argc < 2) {
        std::cerr << "Usage: " << argv[0] << " <input_file_name>" << std::endl;
        return 1;
    }

    // Attempt to open command input file
    std::ifstream file(argv[1]);

    if (!file.is_open()) {
        std::cerr << "Error: Could not open file " << argv[1] << std::endl;
        return 1;
    }

    MaxHeap emailQueue; // Instantiate custom max-heap
    std::string line;    // Variable to hold each line read from file

    // Process file line by line
    while (std::getline(file, line)) {
        // Skip empty lines or carriage returns
        if (line.empty() || line == "\r") {
            continue;
        }

        // Handle EMAIL Command
        if (line.rfind("EMAIL ", 0) == 0) {
            // Extract content after "EMAIL "
            std::string emailData = line.substr(6);
            std::stringstream ss(emailData);

            std::string category, subject, dateStr;

            // Parse category, subject line, and date separated by commas
            if (std::getline(ss, category, ',') &&
                std::getline(ss, subject, ',') &&
                std::getline(ss, dateStr)) {

                // Trim lingering carriage returns from line endings if present
                if (!dateStr.empty() && dateStr.back() == '\r') {
                    dateStr.pop_back();
                }

                // Construct Email object and push into Heap
                Email newEmail(category, subject, dateStr);
                emailQueue.insert(newEmail);
            }
        }

        // Handle NEXT Command
        else if (line == "NEXT" || line == "NEXT\r") {
            // Safely handle NEXT command on empty or non-empty heap
            if (emailQueue.isEmpty()) {
                std::cout << "No unread emails." << std::endl;
            }
        }
    }
}

```

```

    } else {
        Email topEmail = emailQueue.peek();
        std::cout << "Next email:" << std::endl;
        std::cout << "Sender: " << topEmail.getSenderCategory() << std::endl;
        std::cout << "Subject: " << topEmail.getSubject() << std::endl;
        std::cout << "Date: " << topEmail.getRawDateStr() << std::endl;
    }
}
// Handle READ Command
else if (line == "READ" || line == "READ\r") {
    // Safely handle READ command on empty or non-empty heap
    if (emailQueue.isEmpty()) {
        std::cout << "No emails to read." << std::endl;
    } else {
        emailQueue.extractMax(); // Extract and discard highest-priority email
    }
}
// Handle COUNT Command
else if (line == "COUNT" || line == "COUNT\r") {
    // Display total unread count
    std::cout << "There are " << emailQueue.size() << " emails to read." << std::endl;
}
}

file.close(); // Close input file stream
return 0;     // Successful execution
}

```